AI PROJECT · SPRING 2026

Draw & Solve

Handwritten Math Expression Recognition

Matti H. V. Lehmann
Introduction · Conclusion

David A. Spickenheuer
System Pipeline

Jonas Schenke
Image Processing

Philip Nguyen
Dataset & Data Prep

Staniya Thomas
CNN & Evaluation

Abhirup Sain
Live Demo

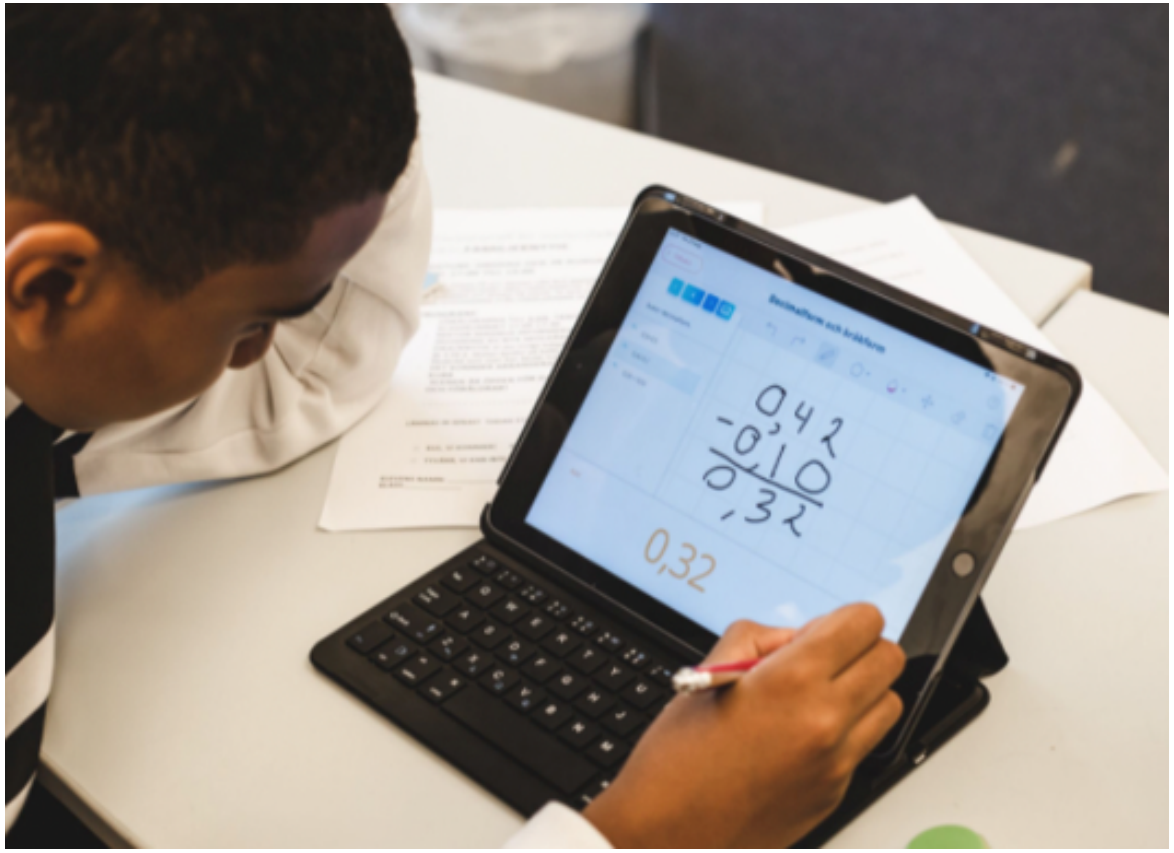
Team & Task Division

Name	Student ID	Task
Matti Hannes Vincent Lehmann	T14H06309	Introduction · Conclusion
David Axel Spickenheuer	T14H06329	System Pipeline
Jonas Schenke	T14H06307	Image Processing & Segmentation
Philip Nguyen	91499153X	Dataset & Data Preparation
Staniya Thomas	T14H06310	CNN Architecture & Evaluation
Abhirup Sain	T14H06318	Live Demo

Motivation

- **Handwritten maths** on tablets and touch screens is increasingly common (online exams, digital whiteboards, teaching apps)
- **Manual typing** of expressions into a calculator or CAS is slow and error-prone
- **Goal**: draw an expression once and obtain an instant, reliable numerical result

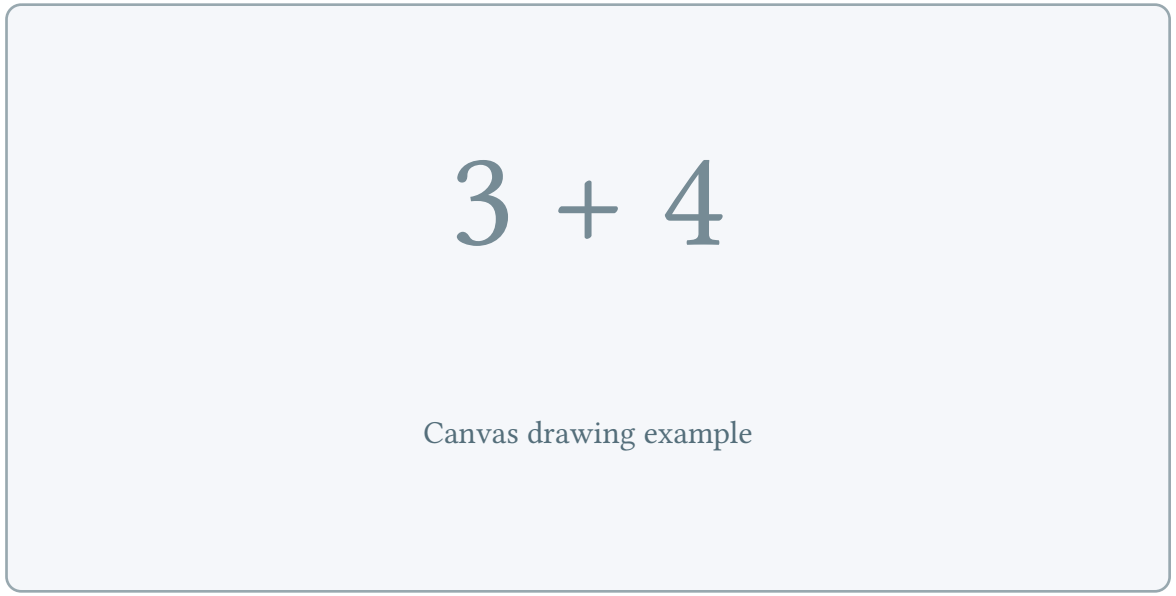
> **Why now?** Tablet adoption in education has grown rapidly — a seamless, pen-first math solver removes a major friction point for students and teachers.



The Problem

- Handwriting varies widely per writer
- Operators share features with digits (e.g. × vs x, − vs ÷)
- Multi-stroke digits confuse naive segmenters

Goal: Draw a math expression on screen ⟶ get the result instantly.



Our Solution: Draw & Solve

1. Draw an expression on a browser canvas
2. OpenCV segments individual symbols
3. CNN classifies each symbol (14 classes)
4. Safe AST evaluator computes the result

Digits

0 · 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9

Operators

+ · − · × · ÷

App screenshot

v1 → v2: What Changed

Aspect	v1	v2
Classifier	MNIST CNN + geometry heuristics	**Unified 14-class CNN**
Dataset	MNIST only	Sagyamthapa (Kaggle)
Accuracy	99% digits · operators fragile	**99.44%** all 14 classes

One model for everything — no hand-crafted rules for operators, just learned features.

System Pipeline

1 Canvas
Draw with mouse/touch

→

2 Segment
OpenCV bounding boxes

→

3 Classify
CNN ⟶ softmax(14)

→

4 Assemble
Sort x ⟶ string

→

5 Evaluate
AST safe-eval

Example — user draws “3 + 4”:

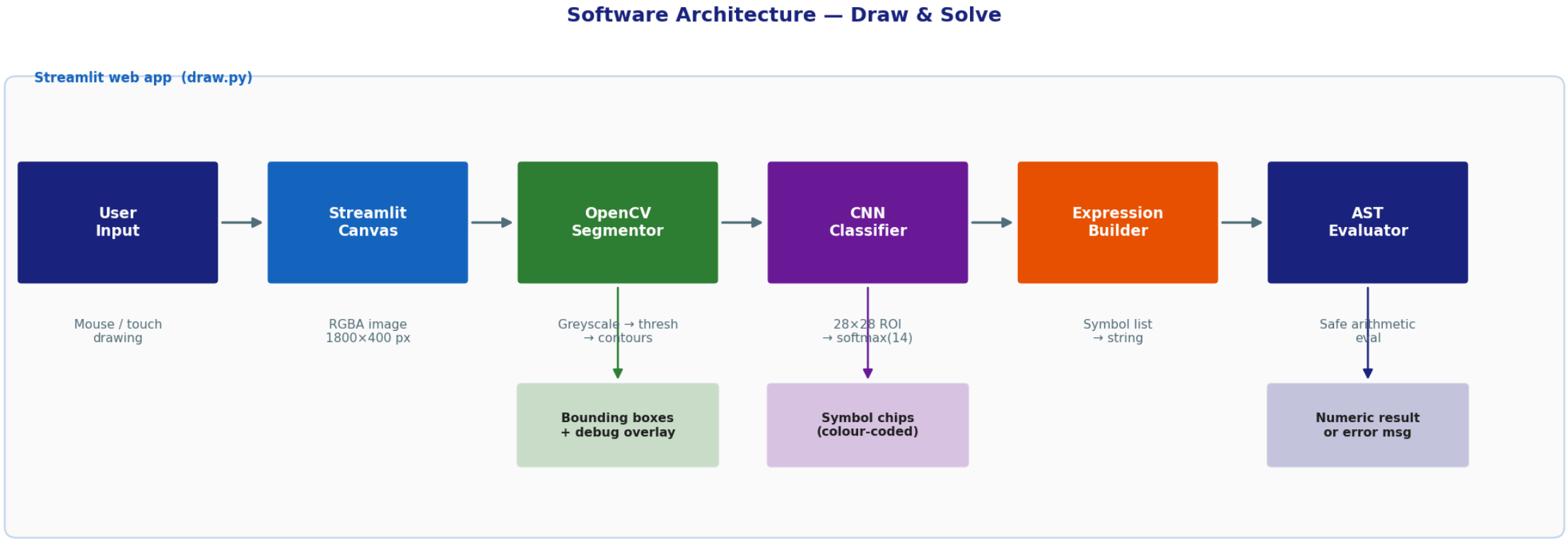
🖍️ “3 + 4” on canvas → 3 bounding boxes → “3” · “+” · “4” → `"3+4"` → **7**

🖥️ **Frontend**
HTML5 Canvas · Streamlit

🔍 **Segmentation**
OpenCV · proximity merge

🧮 **Evaluation**
AST safe-eval · no exec()

Software Architecture



Frontend

Streamlit · HTML5 Canvas

Backend

Python · OpenCV · TensorFlow

Model

Keras · 14-class CNN

Preprocessing & Inference

Per-symbol preprocessing

- Crop bounding box from greyscale image
- Add white padding, scale to fit 26 × 26
- Centre on 28 × 28 white canvas
- Divide ÷ 255 ⟶ ink ≈ 0, background ≈ 1

Inference & assembly

- CNN softmax ⟶ 14 class probabilities
- Confidence < 0.60 ⟶ label as ?
- Sort symbols by x ⟶ expression string
- AST evaluator: only +, −, ×, ÷ permitted (no `eval()`)

Raw ⟶ ROI ⟶ 28 × 28 model input

Thresholding & Contour Detection

Binarisation

- Canvas: white (255), ink ≈ 0
- `THRESH_BINARY_INV`: ink ⟶ 255, bg ⟶ 0
- 3 × 3 dilation closes gaps between strokes
- `findContours` detects white regions on black

Bounding boxes

- Outermost contours only (`RETR_EXTERNAL`)
- Box area < 30 px² discarded (noise filter)
- Sort all boxes left ⟶ right by x-coordinate

Gray ⟶ threshold ⟶ dilated ⟶ boxes

Proximity Merging

Problem: digits 4, 5, 7 are multi-stroke → each stroke gets its own box

Solution: merge horizontally adjacent boxes with gap ≤ **20 px**

- Iterate boxes left → right
- If next box starts within 20 px, expand current box
- Otherwise, emit current box and start a new one

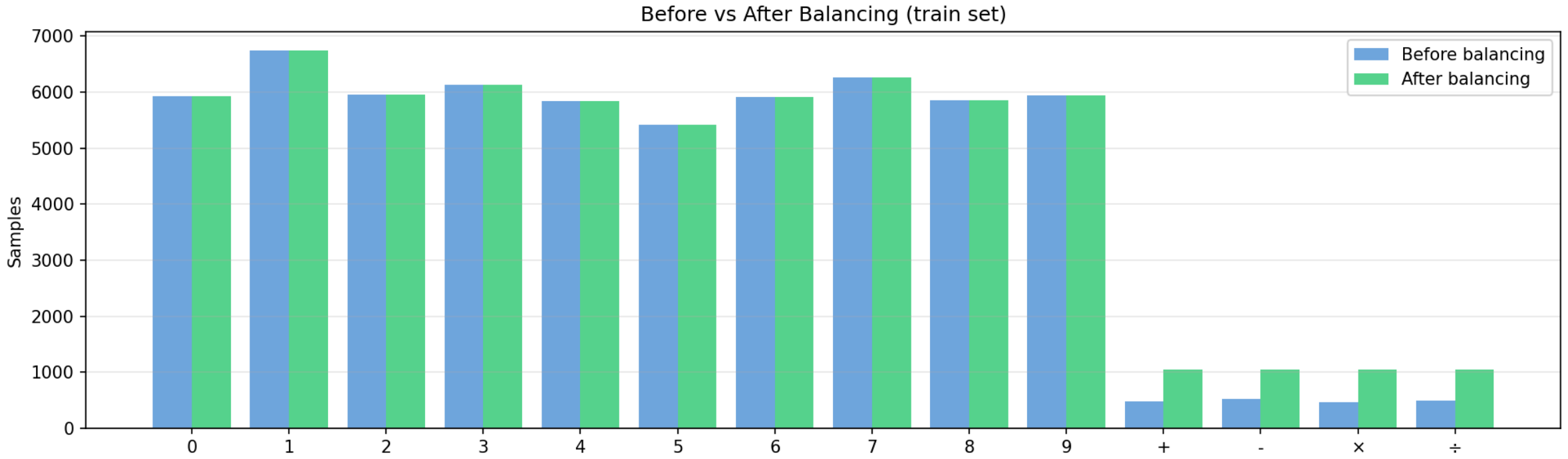
> Also handles the two dots in ÷ after dilation merges them with the bar.

Before / after proximity merge

Dataset & Train/Test Split

Source	Detail
Digits 0–9	MNIST (LeCun et al.)
Operators +−×÷	Sagyamthapa (Kaggle)

Classes used	**14** (digits + operators)
Balanced per class	**1 048 samples**
Total training pool	14 672 images
Train / Val	85 % / 15 % of pool
Test set (held-out)	**10 490 images**
— MNIST test	10 000 images
— Kaggle ops (20 %)	490 images (seed = 42)



Class Balancing & Augmentation

Imbalance problem

- Digits: 5 000 – 6 700 samples each
- Operators: 1 048 samples each ⟶ **6:1 ratio**
- Without fix: model predicts digits almost always

Fix: downsample digits to 1 048 per class

Result: **perfectly balanced** — 1 048 × 14 = 14 672 images

Augmentation (training only, fill = black)

Transform	Range
Rotation	± 8°
Zoom	± 10%
Translation	± 8% x/y

- Simulates real writing variation
- Fill = **0 (black)** — matches MNIST convention
- Key to generalizing from only 1 048 samples